🚀 DAY 1: PROJECT SETUP - LET'S DO THIS TOGETHER!

Alright! Let's start from **absolute zero** and build your project foundation TODAY. I'll guide you through every single step like I'm sitting right next to you.

📌 DAY 1 GOAL

By the end of today, you will have:

- ✅ All software installed
- ✅ Spring Boot project running
- ✅ Database connected
- ✅ First API working
- ✅ Code on GitHub

Time needed: 2-3 hours

🎯 STEP 1: INSTALL REQUIRED SOFTWARE (30 minutes)

1.1 Install Java JDK 17

Why? Java is what runs your Spring Boot application.

How to install:

For Windows:

1. Go to: https://www.oracle.com/java/technologies/downloads/#java17
2. Download "Windows x64 Installer"
3. Run the installer (keep clicking Next)
4. After installation, verify:

Open **Command Prompt** (search "cmd" in Windows) and type:

bash

```
java -version
```

You should see something like: java version "17.0.x"

For Mac:

bash

```
brew install openjdk@17
```

For Linux:

bash

```
sudo apt update

sudo apt install openjdk-17-jdk
```

✅ **Checkpoint:** Can you see Java version when you type java -version?

1.2 Install IntelliJ IDEA Community Edition

Why? This is the best IDE for Java development.

How to install:

1. Go to: https://www.jetbrains.com/idea/download/
2. Download **Community Edition** (it's FREE!)
3. Install it (accept all defaults)
4. Open IntelliJ IDEA

First time opening? It will ask you to:

- Choose theme (Dark or Light - your choice!)
- Skip all plugins for now

✅ **Checkpoint:** IntelliJ IDEA opens successfully?

1.3 Install MySQL

Why? This is your database where all data will be stored.

How to install:

For Windows:

1. Go to: https://dev.mysql.com/downloads/installer/
2. Download "MySQL Installer for Windows"
3. Run installer
4. Choose "Developer Default"
5. **IMPORTANT:** When it asks for root password, set it to: root123 (remember this!)
6. Click Next until it finishes

For Mac:

```
bash

brew install mysql

brew services start mysql

mysql_secure_installation
```

For Linux:

bash

sudo apt update

sudo apt install mysql-server

sudo mysql_secure_installation

After installation, test MySQL:

Open Command Prompt/Terminal:

bash

mysql -u root -p

Enter password: root123

You should see: mysql>

Type: exit to leave MySQL.

✅ **Checkpoint:** Can you login to MySQL?

1.4 Install MySQL Workbench (Optional but Recommended)

Why? Visual tool to see your database.

1. Go to: https://dev.mysql.com/downloads/workbench/
2. Download and install
3. Open MySQL Workbench
4. Click on "Local instance MySQL80"
5. Enter password: root123

✅ **Checkpoint:** Can you see MySQL Workbench interface?

1.5 Install Postman

Why? To test your APIs.

1. Go to: https://www.postman.com/downloads/
2. Download and install
3. Open Postman
4. Skip sign-in (click "Skip and go to app")

✅ **Checkpoint:** Postman opens successfully?

1.6 Install Git

Why? For version control and GitHub.

For Windows:

1. Go to: https://git-scm.com/download/win
2. Download and install (accept all defaults)

For Mac:

bash

brew install git

For Linux:

bash

sudo apt install git

Test Git:

bash

git --version

```

✅ **Checkpoint:** See Git version?

---

## 🎯 STEP 2: CREATE SPRING BOOT PROJECT (20 minutes)

### **2.1 Go to Spring Initializr**

1. Open browser: https://start.spring.io/

You'll see a form. **Configure EXACTLY like this:**

**Project Settings:**

- Project: **Maven** (keep default)

- Language: **Java**
```

- Spring Boot: **3.2.0** (or latest 3.x version)

- Packaging: **Jar**

- Java: **17**

Project Metadata:

- Group: `com.suhada`

- Artifact: `furniture`

- Name: `furniture`

- Description: `Suhada Furniture Management System`

- Package name: `com.suhada.furniture`

Dependencies (Click "ADD DEPENDENCIES"):

Search and add these ONE BY ONE:

1. **Spring Web** (for REST APIs)

2. **Spring Data JPA** (for database)

3. **MySQL Driver** (to connect MySQL)

4. **Lombok** (to reduce code)

5. **Validation** (for input validation)

6. **Spring Boot DevTools** (for auto-reload)

2.2 Generate Project

1. Click green **"GENERATE"** button at the bottom

2. A file `furniture.zip` will download

3. Extract this zip file to a good location, like:

  - Windows: `C:\Users\YourName\Projects\furniture`

  - Mac/Linux: `~/Projects/furniture`

✅ **Checkpoint:** You have a folder named `furniture` with files inside?

🎯 STEP 3: OPEN PROJECT IN INTELLIJ (10 minutes)

3.1 Import Project

1. Open **IntelliJ IDEA**
2. Click **"Open"**
3. Navigate to your extracted `furniture` folder
4. Select the **folder** (not any file inside)
5. Click **OK**

3.2 Wait for Dependencies

IMPORTANT: Bottom right corner will show "Importing..."

This can take **5-10 minutes** the first time!

IntelliJ is downloading all Spring Boot libraries.

☕ **Take a break, grab water!**

3.3 Verify Project Structure

On the left side, you should see:

```
furniture
├── src
│   └── main
│       └── java
│           └── com.suhada.furniture
```

```
│           └── FurnitureApplication.java
│     └── resources
│         └── application.properties
├── pom.xml
```

✅ **Checkpoint:** Can you see this structure?

🎯 **STEP 4: CREATE DATABASE (5 minutes)**

4.1 Create Database

Open **MySQL Workbench** (or Command Prompt with MySQL)

In MySQL Workbench:

1. Click "Local instance"
2. Enter password: root123
3. In the query window, type:

sql

CREATE DATABASE suhada_furniture;

4. Click the **lightning bolt** icon (or press Ctrl+Enter)
5. You should see: "1 row(s) affected"

In Command Prompt:

bash

mysql -u root -p

 # Enter password: root123

CREATE DATABASE suhada_furniture;

exit

` ` `

✅ **Checkpoint:** Database created without errors?

🎯 STEP 5: CONFIGURE DATABASE CONNECTION (10 minutes)

5.1 Open application.properties

In IntelliJ, navigate to:

```

src/main/resources/application.properties

It's probably empty. **Delete everything** and paste this:

properties

# Server Configuration

server.port=8080


# Database Configuration

spring.datasource.url=jdbc:mysql://localhost:3306/suhada_furniture

spring.datasource.username=root

spring.datasource.password=root123

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver


# JPA Configuration

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect


# Logging

logging.level.com.suhada.furniture=DEBUG
```


IMPORTANT: Change `spring.datasource.password=root123` to YOUR MySQL password if it's different!

Press **Ctrl+S** (Windows) or **Cmd+S** (Mac) to save.

✅ **Checkpoint:** File saved without errors?

🎯 STEP 6: RUN YOUR FIRST APPLICATION (10 minutes)

6.1 Find FurnitureApplication.java

Navigate to:

```
src/main/java/com/suhada/furniture/FurnitureApplication.java
```

It should look like this:

```java
package com.suhada.furniture;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FurnitureApplication {

    public static void main(String[] args) {
        SpringApplication.run(FurnitureApplication.class, args);
    }

}
```

6.2 Run the Application

Method 1: Click the **green play button** (▶) next to `public static void main`

Method 2: Right-click on the file → **Run 'FurnitureApplication'**

6.3 Watch the Console

Bottom of IntelliJ, you'll see logs appearing...

Wait for this line:
```
Started FurnitureApplication in X.XXX seconds
```

🎉 **SUCCESS!** Your application is running!

✅ **Checkpoint:** See "Started FurnitureApplication" message?

🎯 STEP 7: CREATE YOUR FIRST API (15 minutes)

7.1 Create Package Structure

In IntelliJ, right-click on `com.suhada.furniture` → **New** → **Package**

Create these packages:

1. `controller`
2. `service`
3. `repository`
4. `entity`
5. `dto`

Your structure should now look like:

```
com.suhada.furniture
├── controller
├── dto
├── entity
├── repository
├── service
└── FurnitureApplication.java
```

7.2 Create TestController

Right-click on controller → **New** → **Java Class**

Name it: TestController

Paste this code:

```java
package com.suhada.furniture.controller;

import org.springframework.web.bind.annotation.*;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/test")
public class TestController {

    @GetMapping("/hello")
    public String hello() {
        return "🎉 Suhada Furniture Backend is Running! 🚀";
    }
```

```
    @GetMapping("/status")
    public Map<String, String> status() {
        Map<String, String> response = new HashMap<>();
        response.put("status", "online");
        response.put("message", "System is working perfectly!");
        response.put("timestamp", LocalDateTime.now().toString());
        response.put("database", "connected");
        return response;
    }

    @GetMapping("/welcome")
    public Map<String, Object> welcome() {
        Map<String, Object> response = new HashMap<>();
        response.put("project", "Suhada Furniture");
        response.put("developer", "Your Name"); // Put your name!
        response.put("version", "1.0.0");
        response.put("features", new String[]{
            "Product Management",
            "Order Processing",
            "AI Assistant",
            "WhatsApp Integration",
            "Real-time Notifications"
        });
        return response;
    }
}
```

Save the file (Ctrl+S)

7.3 Restart Application

Click the **red square** (■) at the top to stop the app.

Click the **green play button** (▶) to restart.

Wait for: `Started FurnitureApplication...`

✅ **Checkpoint:** Application restarted successfully?

🎯 STEP 8: TEST YOUR API WITH POSTMAN (15 minutes)

8.1 Open Postman

8.2 Test Hello Endpoint

1. Click **"New"** → **"HTTP Request"**
2. Method: **GET** (dropdown on left)
3. URL: `http://localhost:8080/api/test/hello`
4. Click **"Send"**

You should see:

```
🎉 Suhada Furniture Backend is Running! 🚀
```

8.3 Test Status Endpoint

1. New request → **GET**
2. URL: http://localhost:8080/api/test/status
3. Click **"Send"**

You should see JSON:

```
json
```

```
{
  "status": "online",
  "message": "System is working perfectly!",
  "timestamp": "2025-01-15T10:30:00",
  "database": "connected"
}
```

8.4 Test Welcome Endpoint

1. New request → **GET**
2. URL: http://localhost:8080/api/test/welcome
3. Click **"Send"**

You should see:

```
json
{
  "project": "Suhada Furniture",
  "developer": "Your Name",
  "version": "1.0.0",
  "features": [
    "Product Management",
    "Order Processing",
    "AI Assistant",
    "WhatsApp Integration",
    "Real-time Notifications"
  ]
}
```

🎉 **AMAZING!** Your first APIs are working!

✅ **Checkpoint:** All 3 APIs returning data?

🎯 STEP 9: SETUP GIT & GITHUB (20 minutes)

9.1 Create .gitignore

In the root `furniture` folder, create a new file called `.gitignore`

Add this content:

```
target/
!.mvn/wrapper/maven-wrapper.jar
!**/src/main/**/target/
!**/src/test/**/target/

### IntelliJ IDEA ###
.idea
*.iws
*.iml
*.ipr

### VS Code ###
.vscode/

### Application properties (if sensitive) ###
# application.properties
```

9.2 Initialize Git

Open **Terminal** in IntelliJ (bottom bar, click "Terminal")

Type these commands ONE BY ONE:

```
bash
git init
git add .
```

git commit -m "Initial commit - Suhada Furniture project setup"

9.3 Create GitHub Repository

1. Go to: https://github.com
2. Sign in (or create account if you don't have one)
3. Click **"+"** at top right → **"New repository"**
4. Repository name: suhada-furniture
5. Description: AI-powered furniture management system
6. Keep it **Public** (for portfolio!)
7. **DON'T** check "Initialize with README"
8. Click **"Create repository"**

9.4 Push to GitHub

Copy the commands GitHub shows (should look like this):

bash

```
git remote add origin https://github.com/YOUR_USERNAME/suhada-furniture.git
git branch -M main
git push -u origin main
```

Paste them in IntelliJ Terminal and press Enter.

First time? GitHub will ask for credentials. Use your GitHub username and password.

✅ **Checkpoint:** Code is on GitHub?

🎊 **DAY 1 COMPLETE! CELEBRATION TIME!**

✅ **WHAT YOU ACCOMPLISHED TODAY:**

1. ✅ Installed all required software
2. ✅ Created Spring Boot project
3. ✅ Connected to MySQL database
4. ✅ Built folder structure
5. ✅ Created 3 working REST APIs
6. ✅ Tested APIs with Postman
7. ✅ Pushed code to GitHub

🏆 **YOU ARE NOW A BACKEND DEVELOPER!**

📸 TAKE A SCREENSHOT!

For your portfolio:

- Screenshot of Postman showing your API response
- Screenshot of IntelliJ with your code
- Share your GitHub repo link

🚀 WHAT'S NEXT? (Tomorrow - Day 2)

Tomorrow we'll build:

- **User Entity** (for login system)
- **Product Entity** (furniture items)
- **Customer Entity** (who buys furniture)
- **Order Entity** (purchases)
- All with proper relationships!

Your database will start looking professional!

❓ DID YOU FACE ANY ERRORS?

Tell me if you got stuck at any step! Common issues:

Problem 1: "Port 8080 already in use"

- **Solution:** Stop any other programs using port 8080, or change to 8081 in application.properties

Problem 2: "Could not connect to database"

- **Solution:** Check MySQL is running, check username/password in application.properties

Problem 3: Maven dependencies not downloading

- **Solution:** Right-click pom.xml → Maven → Reload Project

💬 TELL ME:

1. **Did you complete Day 1?** (Yes/No)
2. **Which step was hardest?**
3. **Are you ready for Day 2?**

4. **Any errors you faced?**

Let's celebrate your progress and move to Day 2 tomorrow! 🎉 🚀

Retry